

Risikoanalyse

Bei der Risikoanalyse wurden jegliche Risiken festgehalten, welche im Laufe des Projekts auftreten könnten und die Höhe des Risikos bzw. Die Gewichtung wurde entsprechend bewertet. Anschließend wurden Maßnahmen gegen diese Risiken aufgestellt, um das Auftreten dieser Probleme zu minimieren bzw. Zu eliminieren.

Risiken	Bewertung	Maßnahmen
APIs fordern unter Umständen eine Anfrage zur Nutzung dieser/ müssen passende ÖPNV API finden, welche so viele Anforderungen von uns abdeckt wie möglich.	mäßig	mehr Zeit einplanen für den Prozess der Recherche und der Beschaffung.
Die Anwendung greift auf verschiedene APIs zu, wie der Deutsche Bahn API. Haltestellen-, Fahrzeiten- und Zugausfall Daten werden benötigt, damit die Anwendung richtig funktionieren kann.	hoch	Frühzeitig abklären, welche APIs für die Nutzung im Projekt zugänglich sind, um ggf. Alternativen zu suchen oder eine eigenständige Datenbank mit den fehlenden Daten anlegen zu können(falls möglich)
Da das Team unterschiedliche Kenntnisse der Programmiersprachen beherrscht, könnte es sein, dass die effizienteste Programmiersprache für die entsprechende Anwendung nicht jedes Projektmitglied gleich gut kann, was bei Neuerwerb einer Sprache zu zeitlichen Verzögerungen führen könnte.	mäßig	Eine geeignete Programmiersprache finden, die beide Projektpartner beherrschen
Netzwerk kreieren - setzt viele Nutzer voraus	mäßig	Ist für den Prototyp erstmal irrelevant, da wir keine echten Nutzer auf die App zugreifen lassen. Würde man die Anwendung auf den Markt bringen, müsste man mit gezieltem Marketing versuchen, Nutzer zu

		gewinnen
Wir setzen auf die soziale Bereitschaft der Nutzer, freiwillig jemanden mit ihrem Ticket mit zu nehmen	mäßig	Ist für den Prototyp erstmal irrelevant, da wir keine echten Nutzer auf die App zugreifen lassen. Würde man die Anwendung auf den Markt bringen, müsste man auch hier einen Reiz für die Personen mit Ticket finden, die Anwendung zu nutzen.
Bei der Entwicklung der Anwendung könnten fehlerhafte Funktionalitäten entstehen, die entweder zu Bugs oder Show-Stopper führen. Diese können gefährlich sein, je nachdem wann sie gefunden werden.	hoch	Test Driven Development orientiert zu arbeiten oder zumindest pro Funktionalität ein paar Tests schreiben. So werden Bugs bzw. Show-Stopper bis zu einem bestimmten Grad vorgebeugt.
Das Konzept könnte von den zuständigen Unternehmen nicht als gute Idee gesehen werden, da diese theoretisch auch Kundschaft verringern kann.	mäßig	Ist für den Prototyp erstmal nicht wichtig. Falls die Anwendung es auf den Markt schafft, würde dieser Punkt ein höheres Risiko darstellen.
Ausfall einer der Teammitglieder	hoch	Regelmäßige Updates geben, strukturiert zu arbeiten, gleichmäßige Aufgabenverteilung, Aufgaben in kleinere Häppchen aufteilen.
Genug Nutzer erreichen	hoch	Durch Abwägen auf welchem Endgerät die Anwendung laufen soll und, im Falle einer App, welche Mobilen App-Entwicklungstechnologien genutzt werden sollen.
Große Internetabhängigkeit	hoch	So wenig internetabhängige Funktionen in der Anwendung integrieren wie möglich.